

SEUPD@CLEF: Team RISE

Davide Furlan¹, Giulia Gibellato¹, SeyedehSara NaziriAlhashem¹, Emanuele Pase¹,
Alberto Pasqualetto¹ and Filippo Tiberio¹

¹University of Padua, Italy

Abstract

This report outlines the development of the RISE group’s Information Retrieval (IR) system for the LongEval-WebRetrieval CLEF 2025 Lab. The objective was to design an efficient, scalable search engine capable of handling large-scale collections with a focus on consistent performance. The proposed system incorporates a modular architecture, including a parser, an analyzer, an indexer and a searcher, then also query translation and expansion using the Gemini LLM, and a re-ranking component to enhance retrieval quality. Emphasis was placed on optimizing indexing and searching speed through multi-threading, improving relevance via title and URL-based field boosting and aligning user queries with the predominant language (French) of the document collection. The evaluation has followed a stepwise enhancement approach, beginning with a Lucene-based baseline. Then, removing stoplists and using a specific stemming, the initial nDCG scores have improved. Also query boosting using a crafted title and the URL of the web page, followed by light query expansion, progressively increased nDCG metric.

Keywords

Document Parsing, Query Translation, Query Expansion, Reranking, CLEF 2025, LongEval-WebRetrieval, Information Retrieval, Temporal Evolution, Search Engine

1. Introduction

Enormous amounts of data is published every day online: to make it useful it must be stored, parsed, indexed and retrieved efficiently.

The best way to retrieve such data is through the implementation of search engines: a user asks for an information (aka a query) and the model will return a document (a webpage when talking about websites) which satisfies the most the initial request. Search engines take a fundamental role in information retrieval across many different domains such as entertainment, healthcare, education etc..

Our group’s goal was to build an efficient search engine for the CLEF LongEval-WebRetrieval 2025 collection. In our work we focused on computational speed and scalability, while targeting best results in terms of *Normalized Discounted Cumulated Gain (nDCG)* metric.

The paper is organized as follows: Section 1.1 links to LongEval’s previous years related works; Section 2 describes our approach and the pipeline; Section 3 shows our experimental setup and codebase; Section 4 discusses our results; finally, Section 5 draws some conclusions and outlooks for future work.

1.1. Prior exploration

At the beginning of our work, we spent a few days understanding which innovative or interesting solutions our colleagues from previous years implemented. Below are some information we have gathered:

- **JIHUMING team** [1]: They highlighted the avoidance of *Named Entity Recognition (NER)* due to efficiency issues.

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2024/2025

✉ davide.furlan.10@studenti.unipd.it (D. Furlan); giulia.gibellato@studenti.unipd.it (G. Gibellato);
seyedehsara.naziri.alhashem@studenti.unipd.it (S. NaziriAlhashem); emanuele.pase@studenti.unipd.it (E. Pase);
alberto.pasqualetto.2@studenti.unipd.it (A. Pasqualetto); filippo.tiberio@studenti.unipd.it (F. Tiberio)



© 2025 Copyright for this paper by its authors.

Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

- **NEON team** [2]: Avoidance of WordNet usage for query expansion. Not good results because it only works with texts in English.
- **QEVALS team** [3]: They used OpenAI's GPT 3.5 turbo for query expansion. From this idea, we decided to use Google Gemini for query expansion.
- **IRIS team** [4]: From the IRIS team we took the idea of using an additional document field for the URL, with the purpose of saving the web address of the document in case the query explicitly asked for the URL. Additionally From IRIS we took some ideas about stoplists sets and to use the French Light Stemmer.
- **MOUSE team** [5]: We used the MOUSE team's project, the last year's best performing work coming from UniPD, as a reference of the steps to do and which components can be the best performing ones.

2. Methodology

[Apache Lucene](#) is the core framework we used to build our search engine. It is a high-performance, full-featured text search engine library written in Java. Lucene is widely adopted in the *Information Retrieval (IR)* community. We also implemented [Apache OpenNLP](#) for natural language processing tasks, such as tokenization and *Named Entity Recognition (NER)*, but we did not use it in our final system.

2.1. Analyzer

The pivotal component of a search engine built with Apache Lucene is the analyzer. The analyzer processes text data from both documents and queries before indexing and searching, breaking it down into tokens and applying various transformations or filters to improve the effectiveness of the search.

It is implemented as `RiseAnalyzer`, a Java class that extends the `Analyzer` class from Lucene.

Our implementation of the analyzer is dynamic, which means that it can be changed at runtime without the need to recompile the code. This is achieved by using a JSON configuration file that specifies the components of the analyzer. The configuration file contains a `Tokenizer`, and a list of `TokenFilters`. Both standard (from Lucene) and OpenNLP analyzers are supported. The following is the configuration file for our best performing analyzer:

Listing 1: Configuration file for the analyzer.

```
{
  "tokenizer": {
    "kind": "core",
    "name": "org.apache.lucene.analysis.standard.
      StandardTokenizer"
  },
  "filters": [
    {
      "kind": "class",
      "name": "org.apache.lucene.analysis.miscellaneous.
        ASCIIIFoldingFilter"
    },
    {
      "kind": "class",
      "name": "org.apache.lucene.analysis.LowerCaseFilter"
    },
    {
      "kind": "elision",
      "name": "filters / FR_elision - articles.txt"
    }
  ]
}
```

```

    {
        "kind": "class",
        "name": "org.apache.lucene.analysis.fr.
            FrenchLightStemFilter"
    }
]
}

```

This approach allows us to easily experiment with different analyzers and find the one that works best for our data.

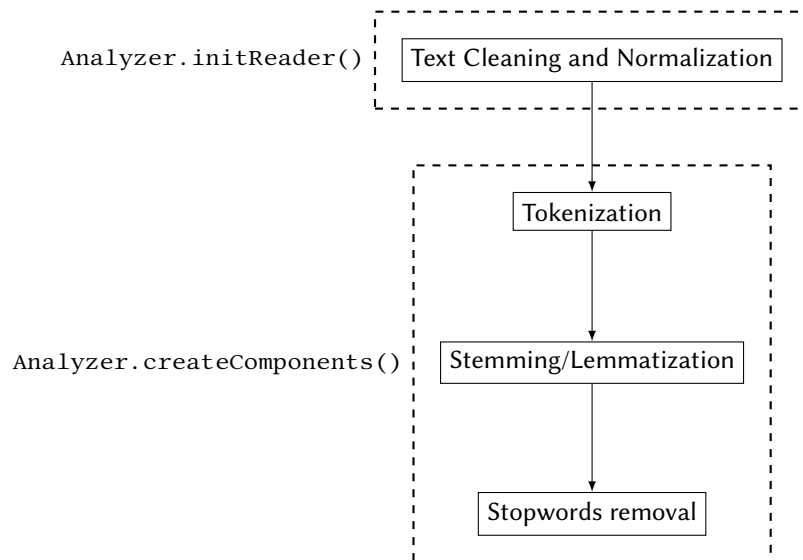


Figure 1: Pipeline of an implementation of Analyzer Lucene Class

2.1.1. Text Cleaning and Normalization

Document cleaning is a crucial preprocessing step in IR-based systems. Before tokenization, the pipeline performs extensive normalization and cleaning on raw text, aiming to standardize information such as *dates*, and *phone numbers* to maximize matching with queries presenting dates or phone numbers, while also removing irrelevant symbols, characters, and few digits numbers.

A modular `TextPreprocessor` Java class has been developed, which performs several transformations:

Date Normalization French and English natural language dates (e.g., *26 janvier 1984*, *February 5th, 1988*) are converted into a unified `dd_mm_yyyy` format using regular expressions and language-specific month mappings. Numeric date formats (e.g., *26 01 1984*) are also normalized to the same structure.

Phone Number Normalization French phone numbers in various formats are standardized to a plain digit format (e.g., *0612345678*) to improve matching and reduce token ambiguity.

Short Number Removal Standalone numeric tokens with 1 to 3 digits that are not part of dates or phone numbers are removed to eliminate noise, which commonly arises from HTML or CSS code coming from the scraping process.

Character Cleanup The pipeline filters out emojis, hashtags, HTML and CSS code, invisible Unicode characters, and styling directives, using a compiled regex pattern.

The cleaning process is fully integrated into the `RiseAnalyzer` class by overriding the `initReader()` method. This ensures that all text passed into the Lucene pipeline is cleaned consistently and efficiently before tokenization.

One particular challenge was preserving semantically meaningful tokens while eliminating noise. To address this, the cleaning module protects normalized entities (dates and phone numbers) during filtering and restores them afterward. This multi-step protection strategy minimizes false deletions.

By adopting this robust cleaning pipeline, both the quality of the index and the consistency of token-level features are improved, which benefits later stages of query matching.

2.1.2. Tokenization

Tokenization is the fundamental step of any Analyzer and its quality significantly affects the performance. We have explored the following tokenizers with the goal of identifying the most suitable one for our pipeline:

WhitespaceTokenizer (from Lucene) it is a simple tokenizer that splits text on whitespace characters. While fast and lightweight, it lacks the sophistication to handle punctuation or language-specific rules.

LetterTokenizer (from Lucene) it splits tokens at non-letter characters. Although slightly more refined than `WhitespaceTokenizer`, it still falls short in treating acronyms, contractions, and numeric data appropriately.

OpenNLPTokenizer (from OpenNLP) these models give more linguistically-aware tokenizers. They need to be configured with French (or English) sentence and token models. While these tokenizers offer accurate segmentation and sentence-level analysis, they introduce significant runtime overhead, and they focus on just one language.

StandardTokenizer (from Lucene) this is a widely adopted tokenizer in the Lucene framework. It provides robust handling of punctuation, alphanumeric strings, email addresses, acronyms and other linguistic patterns using a grammar-based approach. It balances accuracy and performance effectively without the need for external models.

The latter tokenizer is the one we decided to adopt in our pipeline. This decision was guided not only by our empirical observations and the will of keeping indexing step as lightweight as possible, but also by findings in prior reports [1.1]. So some of the key reasons behind this choice include proven reliability, balanced trade-offs and also a sort of community consensus.

2.1.3. Stemming and Lemmatization

The stemmer constitutes the second major step in the offline phase of an IR pipeline. Its primary role is to reduce words to their root forms, enabling the system to match various inflected or derived versions of a word to a common base. This not only helps improve recall by supporting more flexible term matching but also reduces index size. By collapsing word variants into unified representations, stemming introduces a level of semantic normalization that enhances search effectiveness. An example of stemming is the reduction of the words *change*, *changing*, and *changer* to the root form *chang*. Several stemmers like *PorterStemmer* and *SnowballStemmer* are available in the Lucene library.

A similar tool is the lemmatizer, which also reduces words to their base forms but does so transforming words to linguistically correct lemmas. For instance, a lemmatizer would convert *better* to *good* and *went* to *go*. In Lucene lemmatizers are implemented through OpenNLP and they need a language model to work.

Lemmatization is more accurate than stemming, but it is also way more computationally expensive. In the context of our project, where we are dealing with a large collection of documents, we opted for stemming to ensure a faster indexing process.

The chosen stemmer is the `FrenchLightStemmer` [6], which is a stemmer distributed with Lucene and is specifically designed for the French language. The choice of the stemmer has been directed also by previous works by Lorenzo Cazzador, Filippo Galli.

Stoplists Stopwords are very frequent words that do not help discriminate between documents, and they are usually removed from the text before indexing.

We tried implementing different stoplists in our indexing process. We tested both stoplists found on the internet and custom stoplists crafted starting by most frequent words in the corpus.

Since some documents are not in French and, even in French passages, English words may appear, we also considered a stoplist for English words. We have been directed to this approach by the previous work by Filippo Galli.

At first, we used downloaded stoplists to see how it would affect the indexing, but after taking a closer look at the output tokens, we realized that some very common and irrelevant words were still not being filtered out. To fix this, we decided to create some custom stoplist from the most frequent words in the corpus trying to get to a cleaner and smaller index and an improved search result quality.

- `FR_stoplist.txt`: a french stoplist found on the internet. It contains 463 words.
- `customFrenchStoplist.txt`: a custom stoplist created by us merging the most popular stoplists. It contains 552 words.
- `SMART.txt`: the english stoplist of SMART retrieval system [7]. It contains 571 words.
- `50-custom.txt`: a custom stoplist created removing the most frequent 50 words in the corpus.
- `125-custom.txt`: a custom stoplist created removing the most frequent 125 words in the corpus.
- `250-custom.txt`: a custom stoplist created removing the most frequent 250 words in the corpus.
- `500-custom.txt`: a custom stoplist created removing the most frequent 500 words in the corpus.

As we will detail in section 4.1, the use of stoplists has a negative impact on the quality of the results for this dataset.

Another filter we have implemented, similar to stemming is the `ElisionFilter`, which is a Lucene filter that removes French articles and prepositions with apostrophes from the tokens, in the french Language a lot of words are formed by the contraction of a preposition and an article, for example *l'avion* (the plane) will be tokenized as *avion* (plane). To apply such filter we need to provide a list of a few articles to be removed. Those can be found in the `FR_elision-articles.txt` file. We observed this filter to be effective, so we implemented it in our baseline.

2.2. Indexer

To enable efficient retrieval and structured storage of documents, we have developed an indexing component implemented in the `RiseIndexer` class.

This class is responsible for orchestrating the entire indexing workflow: reading raw documents from the parsed corpus, analyzing their textual content through the analyzer, and writing the resulting indexable fields.

In our implementation, the indexing logic is designed to be modular, extensible and optimized for performance through multi-threading. The indexer leverages concurrent workers to process and index documents in parallel, significantly reducing the overall indexing time when handling large corpora.

2.2.1. Language Detector

During the development of the search engine, one of the core challenges we have addressed was the support for multilingual document, specifically in French and English. To this end, we have initially

integrated a language detection component using an OpenNLP model. However, this turned out to be a complex and critical aspect of the system design.

Our idea was to automatically detect the language of each document at indexing time and then apply the most appropriate language-specific analysis pipeline (for example a French analyzer for the French documents and an English analyzer for the English ones). This logic has been encapsulated inside a custom `RiseAnalyzerWrapper` class. The wrapper acts as an abstraction layer over multiple analyzers, dynamically choosing the correct one for each document based on the detected language, without requiring manual separation or prior annotation of the documents.

Despite the theoretical value of this approach, we encountered several practical and architectural challenges:

Thread safety and concurrency integrating dynamic analyzer switching into a multi-threaded indexing pipeline introduced complexity and potential synchronization issues;

Performance concerns given the large number of documents, the overhead introduced by on-the-fly language detection was substantial. Indexing time was observed to increase up to threefold, which significantly impacted efficiency.

To better understand the implications of our design choices, we have run experiments using the language detector along with a set of counters (now commented out in the code), used to track the distribution of languages across documents, identify inconsistencies in language detection for documents sharing the same ID and verify detection accuracy relative to expectations. The results of this analysis revealed that the majority of documents are written in French, with only a small portion in English or other languages.

Based on this observation, and given the performance penalties, we concluded that a fully multilingual indexing pipeline was not worthwhile for this particular dataset.

So we ultimately decided to abandon automatic language detection having also the confirm from the organizers that the task should be French-only, hence non-French documents should be considered. Instead, we have opted for a single-language French pipeline. This allows to maintain high indexing performance and avoid potential inconsistencies in term processing.

On the other hand a language detection and translation component has been applied for the queries. Further details about this design choice are discussed in the part about query translation [2.3.1] in the Searcher section [2.3].

2.2.2. Parser

In the provided collection documents are provided, split in in multiple files, both in JSON and TREC format. They need to be parsed before they can be indexed. At this stage, we also perform an ad hoc cleaning of the text to remove noise such as HTML tags, CSS code, emojis, and other irrelevant characters.

Since the documents were provided to us in two formats, we conducted analyses on the reading speed of both formats to determine which one would be more efficient to reduce as much as possible the indexing time even from the beginning. To do this, the `ParserBenchmark` class has been created to compare the reading speed of TREC and JSON documents, both with handcrafted logic (REGEXes) and using specific libraries. The result showed that the JSON format with manual (handcrafted) reading implementation offers significantly better performance, so we decided to adopt this strategy.

2.2.3. Parsing Speed Testing

Performance evaluations were performed to determine the optimal selection of the parser based on execution speed. To ensure statistical robustness, the parsing times of the complete dataset were recorded across ten independent runs.

Preliminary results demonstrated that parsers employing a manual implementation approach, which avoid external JSON/XML parsing libraries, offered significantly superior performance in terms of

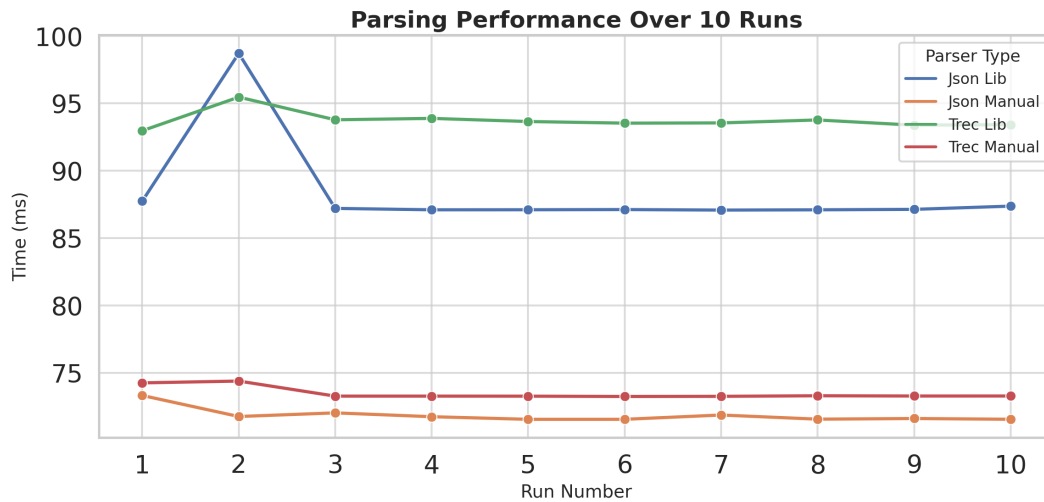


Figure 2: Run/Time plot for the basic parsers

parsing speed. Subsequently, a statistical comparison between parsing TREC and JSON formatted files was carried out using a two-sided Wilcoxon signed rank test. The results indicated a statistically significant speed advantage when parsing JSON files compared to TREC files ($pvalue = 0.001706$).

To further enhance parsing efficiency, a decision was made to develop a dedicated "one-shot" parser, prioritizing processing speed over memory usage. This parser, also operating on JSON-formatted files, fully loads the dataset into memory utilizing the `com.fasterxml.jackson` library and subsequently returns the parsed objects iteratively via the invocation of the iterator's `next()` method. Although this implementation poses potential risks, such as triggering Out-of-Memory (OOM) errors under insufficient system memory conditions, it provides a substantial speed increase beneficial for indexing processes.

A very important class is `ParsedDocument`, which serves to represent the parsed documents, saving:

- **Id:** The numeric identifier value of the document
- **Title:** Crafted from the first part of the document, as described in paragraph 2.2.3
- **Text:** The body of the document, containing the full text

And if present:

- **URL:** The web address of the document
- **Domain:** The domain of the web address
- **Keyword:** Words present within the URL. Used as query expansion to improve document search

Title extraction Examining the documents in the corpus, we found that frequently the first part of the document contains the title of the webpage.

The title is often a concise and informative summary of the content of the document, which makes it a valuable piece of information for the search engine. Hence we decided to exploit this information and extract the title from the document in order to generate a new field in the index called `title` used to boost the score of the documents during the search phase (see Subsection 2.3.2 for more details).

The algorithm we used is simple: it keeps the part of the document that is before a delimiter (" | ", " - ", " ? ", " ! ", " . ") and discards the rest. If the title is not found, the algorithm checks if in the beginning of the document there is a substring all in uppercase letters, which is likely to be the title. If the title is not found, the algorithm keeps the first 70 characters of the document as the title.

A final processing step is to remove duplicates from the title.

The extraction of the title is done in the `TitleExtractor` class.

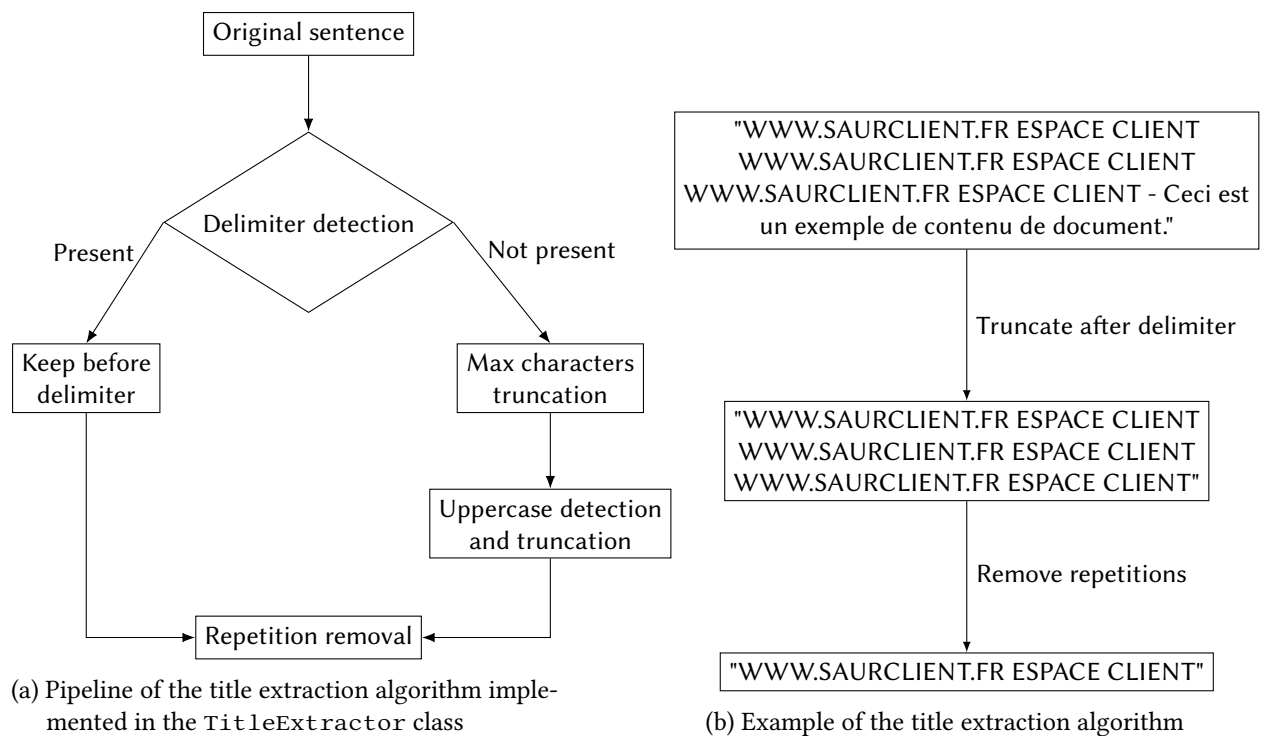


Figure 3: Title extraction algorithm

URL extraction An important feature we developed is the URL extraction. The URL extraction occurs simultaneously with the creation of the `ParsedDocument`: When the document ID is available, the system looks in the `release_2025/collection_db.db` database for the entry with the same ID. Then, it processes the URL by extracting the domain and the words present in the address to save them in the respective fields mentioned above. The main idea behind this decision was to be able to both boost tokens in the query which appear also in the url and to boost authoritative documents which come from a searched domain.

Here is how the URL extraction works in detail:

1. The first step happens during the creation of the `ParsedDocument`, where the ID of the document is used to seek the corresponding *URL*.
2. If an *URL* is found, it undergoes some manipulation to extract the *domain* and the alphanumeric string after the "/" character following the domain itself.
3. This alphanumeric string is further processed to separate the string into substrings, then numbers, special characters, single letters, and file extensions or properties of the page (like ".pdf", ".html", ".php", ".aspx") are removed. We are then left with words that have a length greater than three characters.
4. The final step is to save the *URL* and the *domain* in their relative fields, and the remaining words appending them to the *title* field of the `ParsedDocument`.

We had a couple of options for where to save the words of the URL (i.e., the strings following the domain name):

Storing them in the document body This option was dismissed because the terms would have been treated as regular content words and likely diluted within the broader context of the document, reducing their influence during retrieval.

Storing them in the document title This alternative appeared more promising, as it allowed us to assign greater weight to the terms, thereby increasing their impact on relevance scoring.



Figure 4: URL extraction schema

In the end we did not use the domain field since we observed that the url alignment (described in subsection 2.3.3) was already sufficient to boost the documents coming from the same domain.

DB Access Parallelization Query id to URL mappings appear in the *release_2025/collection_db.db* sqlite database present in the task's dataset.

Initially the `ParsingURL` class took into account the whole reading of the database, but we noticed that this was a huge bottleneck in corpus indexing since database queries were sequential, making vain the effort put into the parallelization of indexing step.

In order to overcome this obstacle, we created the `DBReader` class which provides methods to access the database in a fast and parallelizable way, exploiting a read only approach, mainly by disabling the journal usage, disabling the synchronous mode, and optimizing the database which will be accessed in immutable, read-only mode.

Used Pragmas:

```
PRAGMA mmap_size = 268435456
PRAGMA cache_size = -16000
PRAGMA page_size = 8192
PRAGMA read_uncommitted = 1
PRAGMA synchronous = OFF
PRAGMA journal_mode = OFF
PRAGMA analysis_limit = 1000
PRAGMA optimize
```

A `DBReader` class instance, initialized with a database path, will create multiple `Connections` and `PreparedStatements`, one for each thread, and store them in `ConcurrentHashMaps`

This utility class has been also used for topics reading after query translation (described in subsection 2.3.1) and query expansion (described in subsection 2.3.4) has been implemented for parsing topics and assign them one field each.

2.3. Searcher

The Searcher is the last major component of our information retrieval pipeline. It is responsible for retrieving the relevant documents from the indexed set based on a given query. The main components of the searcher are:

Query formulation (`formulateQuery`) : For each topic, a *BooleanQuery* is created, containing the title and body fields with their respective boosts.

Search execution : This method performs the actual search. `RiseSearcher` is implemented exploiting parallel processing, considerably improving the performance of the system.

The steps involved are:

- For each topic, the Searcher uses the query to retrieve the top documents, ranked by relevance to the query.
- The scores of the retrieved documents are re-ranked.
- The scores are then normalized between 0 and 1 to ensure consistency across queries.
- The results are written to the output file.

Other interesting features of the Searcher are:

- Title boost 2.3.2
- URL Alignment 2.3.3
- Query expansion 2.3.4

2.3.1. French translation of the query

Given that most documents in our dataset are in French, we have implemented a query translation system that converts user queries into French when they are written in another language. This approach helps ensure accurate and consistent retrieval results without requiring a multilingual pipeline.

The translation logic has been implemented in a standalone Python script: `query_translation.py`, this step is performed offline since we exploited the fact we have the full set of queries available in advance inside the `release_2025/queries_db.db` sqlite database.

The pipeline begins by receiving the original query and determining whether translation is necessary. This decision is determined by a prior step: **language detection**.

For this, we used the `FAST_LANGDETECT` Python library. If the detected language is already French (our target language), the system simply bypasses translation and returns the original query. While, if the query is in another language, the pipeline uses the **translation component**, which interfaces with the Google Gemini 2.0 Flash API.

The API is invoked through a custom HTTP POST request, constructed with a carefully tailored French-language prompt that instructs the model to translate the query literally, without interpretation or additional explanation.

The prompt:

Vous êtes un moteur de traduction. Vous recevrez une chaîne de texte arbitraire dans une langue source inconnue.

1. Traduisez le texte mot à mot, en préservant exactement tous les termes et la structure.
2. Traduisez toujours en français.
3. Ne renvoyez que la requête traduite, sans commentaire ni explication.

Voici le texte à traduire: <QUERY>

If translation is successful, the translated query is returned; otherwise, the original query is preserved to ensure no interruption in processing.

This preprocessing pipeline ensures that user queries are consistently aligned with the main language of the indexed documents, improving the accuracy and relevance of retrieval results without introducing significant complexity into the core system.

2.3.2. Title Boost

Since the title field includes both parts of the URL (described in paragraph 2.2.3) and the extracted document title (as described in paragraph 2.2.3), precious information to discriminate relevant documents, we implemented a boost to the terms present in the title field. This boost is applied during the search phase, where the title field is given a higher weight compared to the body field. This means that if a term appears in both the title and the body, it will have a greater impact on the document's score when it appears in the title.

2.3.3. URL Alignment

It was decided to perform a first re-ranking of the documents using the url of the documents: An alignment between the URL and the query was chosen to assess the amount of overlap between the two. Specifically, we decided to use a normalized global alignment parameterized to +1 for matches and -1 for insertions/deletions. In practice, the version based on the dynamic programming technique of the Needleman-Wunsh algorithm has been used: this algorithm calculates the alignment score and normalizes it in the range $[-1, +1]$. The only exception results from the fact that: if the query is an exact substring of the URL, then the score is 1 regardless of the alignment.

Example of our alignment:

```
original url: code.google.com/archive/p/stop-words/  
original query: code google.com stopwords
```

```
Aligned url:  code.-google.com/archive/p/stop-words/
Aligned query: code- google.com-----stop-words-
Score:       +++++-+++++-----+++++-----
```

In order to also parameterize the boost factor $[min_{boost}, max_{boost}]$ during reranking, the score is interpolated by a normalized sigmoidal function centered on zero: the score is first compressed into the range $[0, 1]$, the normalized sigmoidal function is computed, and then extended to the boost factor range. As in the following:

$$\begin{aligned}
t &= \frac{score + 1}{2}, \quad k = 10 \\
f(t) &= \frac{1}{1 + e^{-k(t-0.5)}}, \quad s_0 = \frac{1}{1 + e^{k \cdot 0.5}}, \quad s_1 = \frac{1}{1 + e^{-k \cdot 0.5}} \\
f_{\text{norm}}(t) &= \frac{f(t) - s_0}{s_1 - s_0} \\
f_{\text{final}}(t) &= min_{boost} + f_{\text{norm}}(t) \cdot (max_{boost} - min_{boost})
\end{aligned}$$

Essentially, this approach aims to leverage the semantic content of the URL in order to reward documents with strong alignment and, possibly, slightly penalize those with poor matching.

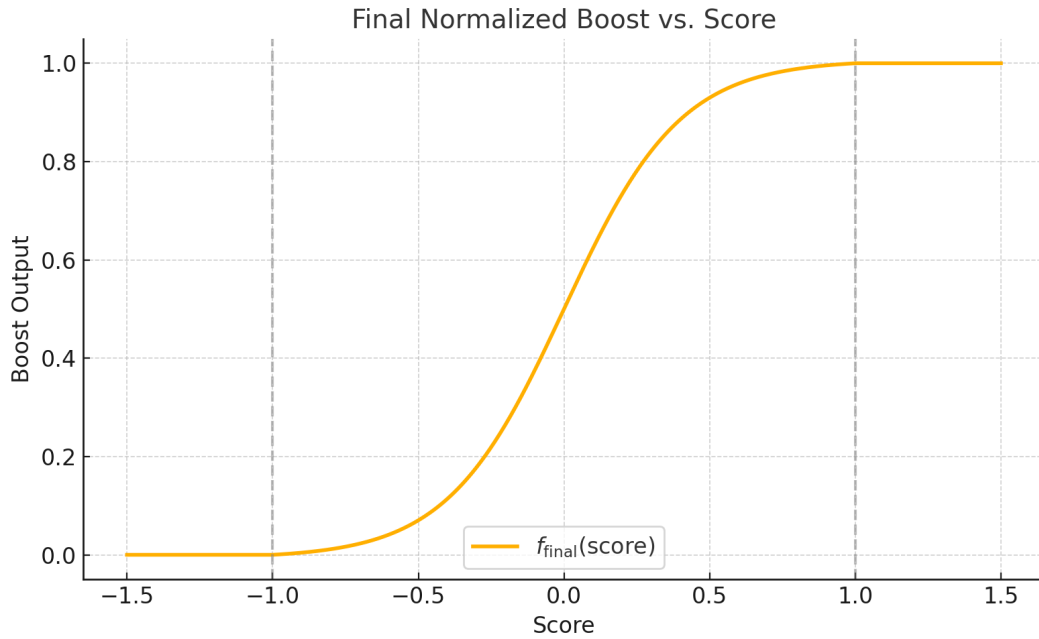


Figure 5: Example of our sigmoid interpolation function for url boosting. Here $k = 10$, $min_{boost} = 0.8$ and $max_{boost} = 1.4$

2.3.4. Query Expansion

We query expand the queries through Google Gemini 2.0 Flash: it permits us to get synonyms and related concepts to the query in analysis. Our prompt to Gemini gives some examples to the model to gain the benefits of few-shot prompting compared to zero-shot one:

Nevertheless, we asked gemini to answer using structured output, so we can easily parse and use it in our pipeline.

The prompt:

Vous faites partie d'un système d'information qui traite les requêtes des utilisateurs. Vous développez une requête donnée en requêtes de sens similaire.

Développez la requête de recherche suivante en une liste de mots-clés et expressions associés. Fournissez la sortie sous forme de tableau JSON de chaînes de caractères. N'incluez aucun autre texte en dehors du tableau JSON.

Exemples :

Requête : recettes de cuisine faciles ["recettes rapides", "cuisine simple", "idées repas faciles", "préparer à manger rapidement", "recettes pour débutants", "meilleures recettes faciles à faire"]

Requête : apprendre le français en ligne ["cours de français en ligne", "sites pour apprendre le français", "applications pour le français", "exercices de français en ligne", "apprentissage du français à distance"]

Requête : <QUERY>

Of particular concern are some queries related to sexual content and violence since they won't be query-expanded even with safety filters turned on. If for this or any other reason the model doesn't return any output, we won't consider any expansion for the query.

3. Experimental Setup

3.1. Used Collections

For the experiments, we employed the training collection TrecEval25 provided by TrecEval. The collection is organized into 9 separate folders, each containing JSON and TREC files. Each folder was treated as an independent dataset - here the longitudinal evaluation that gives the name to the task. Additionally, the provided database files (.db) was utilized to extract the corresponding URLs for each document and to perform query translation and expansion.

3.2. Evaluation Measures

Performance evaluation throughout this work was conducted using the nDCG metric.

$$\text{nDCG}(k) = \frac{\text{DCG}(k)}{\text{iDCG}(k)} \quad (1)$$

where *Discounted Cumulated Gain (DCG)* is computed as:

$$\text{DCG}(k) = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (2)$$

with rel_i representing the graded relevance of the item at position i , and *Ideal Discounted Cumulated Gain (iDCG)* is the DCG(k) value obtained by sorting documents in decreasing order of relevance, thus representing the optimal ranking.

The metrics were extracted from the runs using the *trec_eval* tool (https://github.com/usnistgov/trec_eval).

3.3. Git Repository and its Organization

The entire project, including the source code, generated outputs, analysis, and additional documentation, is available on Bitbucket at the following URL: <https://bitbucket.org/upd-dei-stud-prj/seupd2425-rise/src/master/>

The repository is structured as follows:

- **benchmark**: contains the speed analysis performed on the parsers.
- **code**: contains the source code of the developed system.
- **runs**: contains the runs produced by the developed system.
- **results**: contains the performance scores obtained from the evaluation of the runs.
- **homework-1**: includes the initial report describing applied techniques and insights gained.
- **homework-2**: will contain the final paper submitted to CLEF.
- **slides**: stores the presentation slides related to the conducted project.

3.4. Hardware Used for Experiments

The experiments were conducted on a Windows 10 PC configured as follows:

- **CPU**: Intel Core i5-10400F
- **RAM**: 32 GB DDR4 at 3600 MHz
- **GPU**: NVIDIA GTX 1660
- **Storage**: Dedicated Samsung 990 EVO NVMe SSD

3.5. trec_eval problems

trec_eval software [8] has some problems with files larger than 2GB since on Windows, when compiled, as suggested, with MSVC, functions like lseek output a 32 bit-size value - too little to fit file dimensions larger than 2GB. An ¹issue has been opened on the GitHub repository of trec_eval. Such problem is not present when compiling with GCC on a Linux-based OS.

In order to tackle this issue we reduced the runfile size using a short runid, since the runid can be the longest part of any line:

Anatomy of a runfile line:

```
query_id      Q0      doc_n  rank    score   runid
```

4. Results and Discussion

We analyzed the system by incorporating atomic operation blocks into the pipeline and evaluated the contribution of each individual block when concatenated with the preceding ones. For each block, an optimization phase was carried out to identify the optimal parameters.

4.1. Baseline

We began by attempting to construct a baseline that already yielded satisfactory results and allowed for the predictable integration of subsequent blocks. We evaluated several analyzers and reached the point where it became necessary to test different stoplists in order to determine which could yield the best results.

Following numerous tests, we observed that the nDCG value decreased as the size of the stoplist increased, and that the best performance was achieved when no stoplist was used.

We will therefore define as the BASELINE the "vanilla" Lucene pipeline, which employs the analyzer composed of: `org.apache.lucene.analysis.standard.StandardTokenizer` +

- `org.apache.lucene.analysis.miscellaneous.ASCIIFoldingFilter`
- `org.apache.lucene.analysis.LowerCaseFilter`
- `FR_elision-articles.txt`
- `org.apache.lucene.analysis.fr.FrenchLightStemFilter`

With this setup, we obtained the initial nDCG values in Table 1.

Table 1
nDCG values for BASELINE System

	2022-06	2022-07	2022-08	2022-09	2022-10	2022-11	2022-12	2023-01	2023-02
BASELINE	0.2107	0.2103	0.2200	0.2105	0.3142	0.3168	0.3279	0.3339	0.3014

4.2. Title Boosting

At this stage, we introduced the block responsible for boosting the title extracted from the document.

We tested various multipliers to identify the most effective one: the nDCG values increased up to a multiplier of 0.4x, after which they began to decline as the multiplier continued to increase, as shown in Figures 6, 7 and Table 2.

At this point, we selected 0.4 as the optimal multiplier.

¹Cannot open files bigger than 2GB · Issue #47 · usnistgov/trec_eval

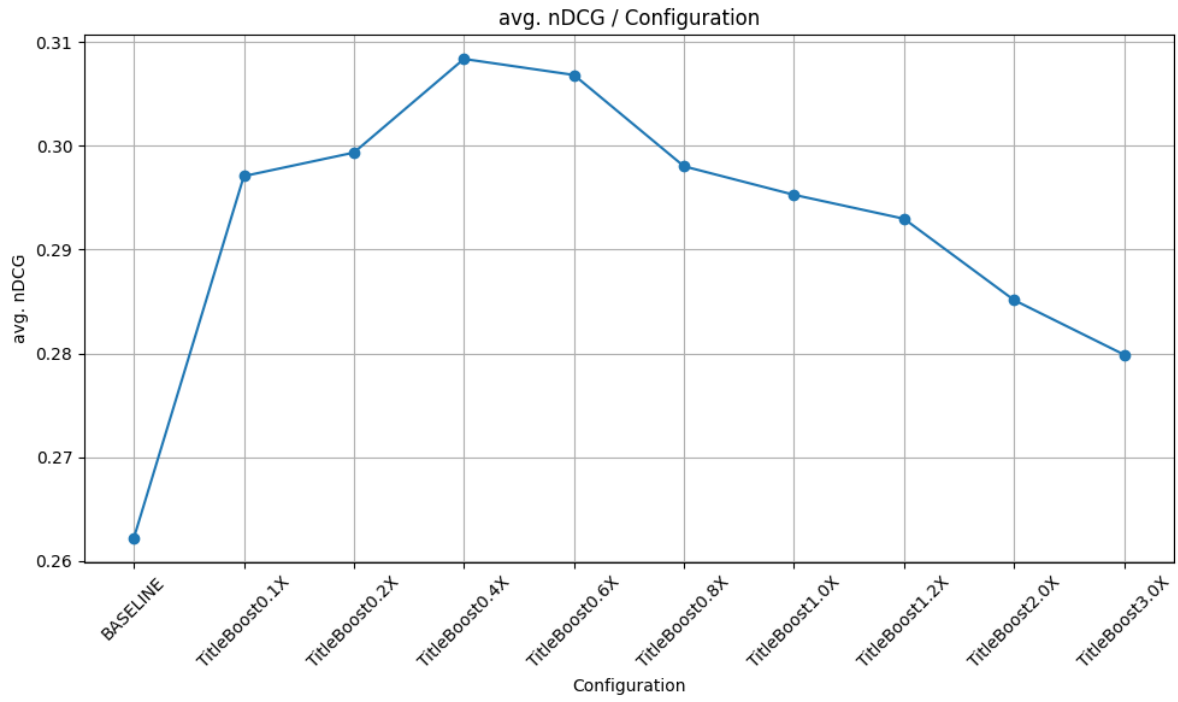


Figure 6: Average nDCG for different title boosting factors

Table 2

nDCG values for BASELINE System vs BASELINE System + different title boosting multipliers

	2022-06	2022-07	2022-08	2022-09	2022-10	2022-11	2022-12	2023-01	2023-02
BASELINE	0.2009	0.2022	0.2118	0.2024	0.3031	0.3061	0.3181	0.3245	0.2908
+TitleBoost0.1X	0.2333	0.2337	0.2398	0.2307	0.3448	0.3463	0.3554	0.3626	0.3271
+TitleBoost0.2X	0.2335	0.2355	0.2428	0.2317	0.3478	0.3495	0.3588	0.3663	0.3282
+TitleBoost0.4X	0.2451	0.2468	0.2499	0.2405	0.3571	0.3573	0.3664	0.3743	0.3379
+TitleBoost0.6X	0.2452	0.2464	0.2483	0.2399	0.3551	0.3554	0.3642	0.3712	0.3357
+TitleBoost0.8X	0.2380	0.2379	0.2406	0.2324	0.3441	0.3452	0.3554	0.3630	0.3256
+TitleBoost1.0X	0.2360	0.2357	0.2376	0.2301	0.3411	0.3425	0.3518	0.3591	0.3236
+TitleBoost1.2X	0.2340	0.2338	0.2352	0.2283	0.3384	0.3394	0.3492	0.3565	0.3219
+TitleBoost2.0X	0.2287	0.2272	0.2285	0.2212	0.3297	0.3298	0.3403	0.3468	0.3140
+TitleBoost3.0X	0.2252	0.2236	0.2235	0.2167	0.3242	0.3237	0.3340	0.3399	0.3081

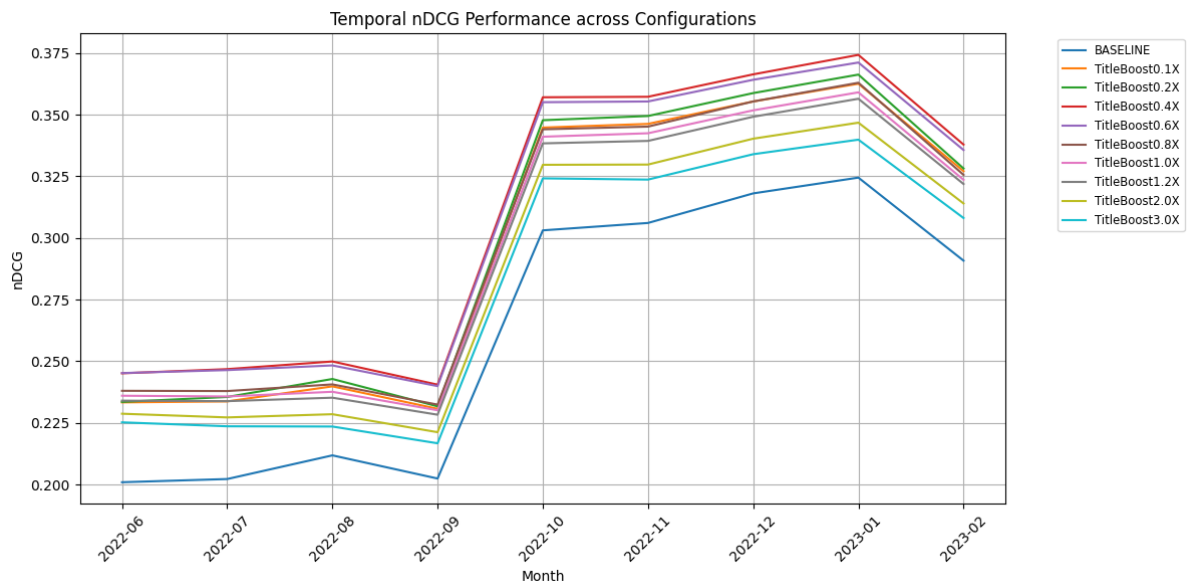


Figure 7: nDCG for different title boosting factors for each month

4.3. URL Alignment Boosting

Subsequently, we integrated into the pipeline the URL Alignment module, which is used to boost or penalize documents based on their alignment score.

Starting from the BASELINE+TitleBoost0.4x system, we tested several boosting intervals for score interpolation and selected the range [0.8, 1.4] which proved to be the best range as shown in Figures 8, 9 and Table 3.

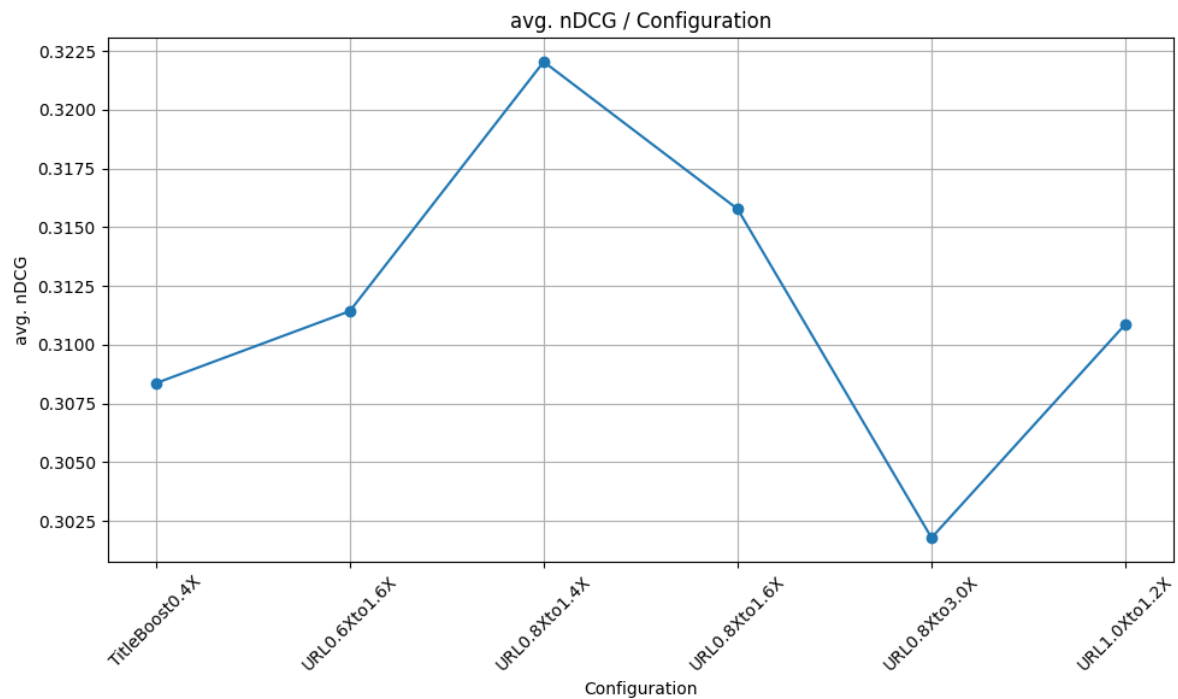


Figure 8: Average nDCG for different URL Boosting Ranges configuration

Table 3

nDCG values for previous optimal pipeline vs previous optimal pipeline + different url boosting

	2022-06	2022-07	2022-08	2022-09	2022-10	2022-11	2022-12	2023-01	2023-02
TitleBoost0.4X	0.2451	0.2468	0.2499	0.2405	0.3571	0.3573	0.3664	0.3743	0.3379
+URL0.6Xto1.6X	0.2518	0.2499	0.2519	0.2429	0.3558	0.3587	0.3687	0.3755	0.3477
+URL0.8Xto1.4X	0.2609	0.2609	0.2625	0.2509	0.3674	0.3704	0.3814	0.3877	0.3563
+URL0.8Xto1.6X	0.2530	0.2531	0.2568	0.2460	0.3613	0.3642	0.3752	0.3825	0.3499
+URL0.8Xto3.0X	0.2451	0.2425	0.2437	0.2355	0.3434	0.3474	0.3562	0.3627	0.3396
+URL1.0Xto1.2X	0.2479	0.2480	0.2533	0.2409	0.3592	0.3606	0.3694	0.3780	0.3406

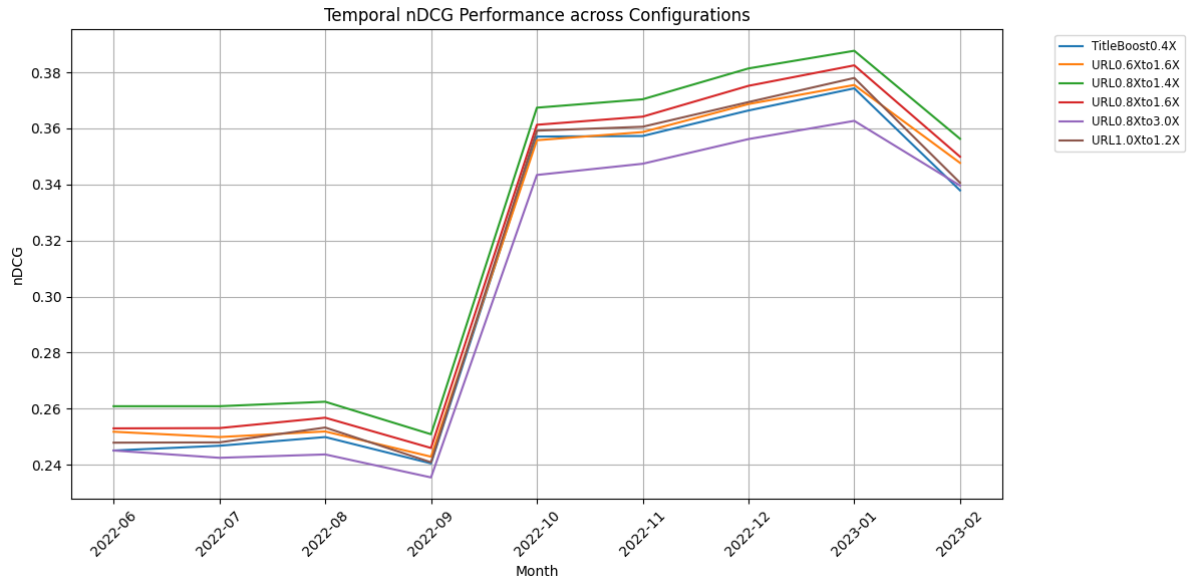


Figure 9: nDCG for different URL Boosting Ranges for each month

4.4. Query Expansion

Subsequently, we integrated the query expansion module into the pipeline.

Starting from the BASELINE+TitleBoost0.4x+URL0.8Xto1.4X system, we tested several boosting factors for expanded queries and selected 0.1 as the optimal one.

The results of this optimization phase are shown in Figures 10, 11 and Table 4.

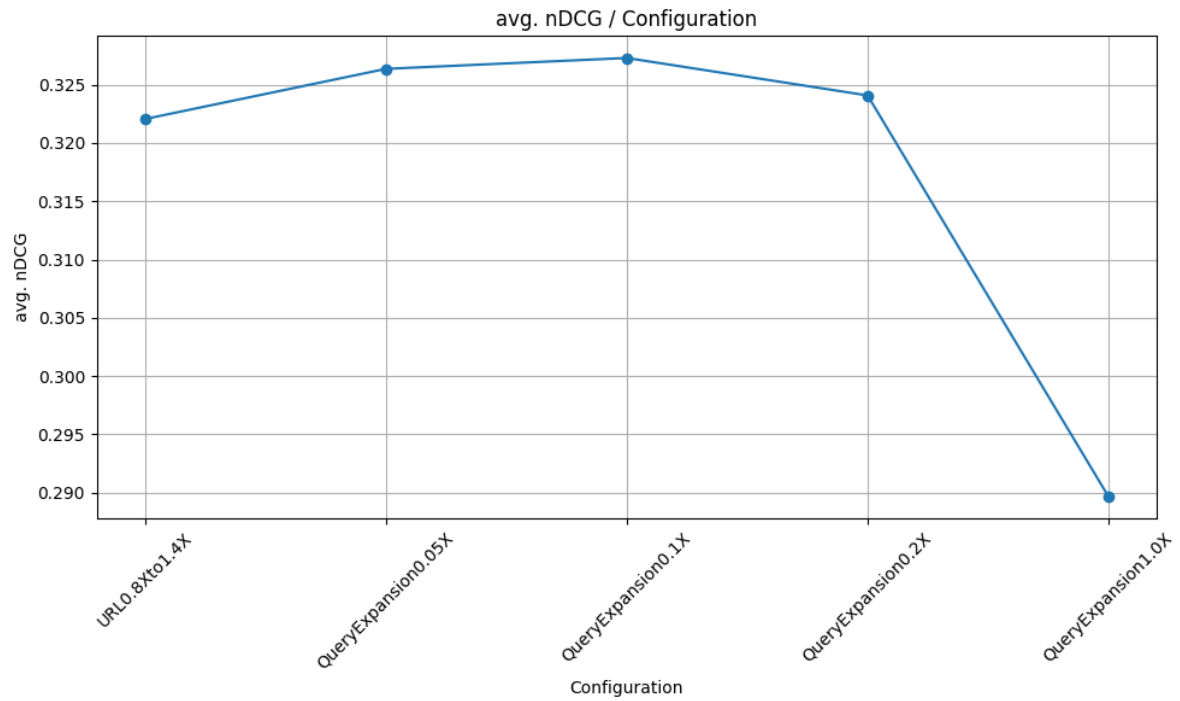
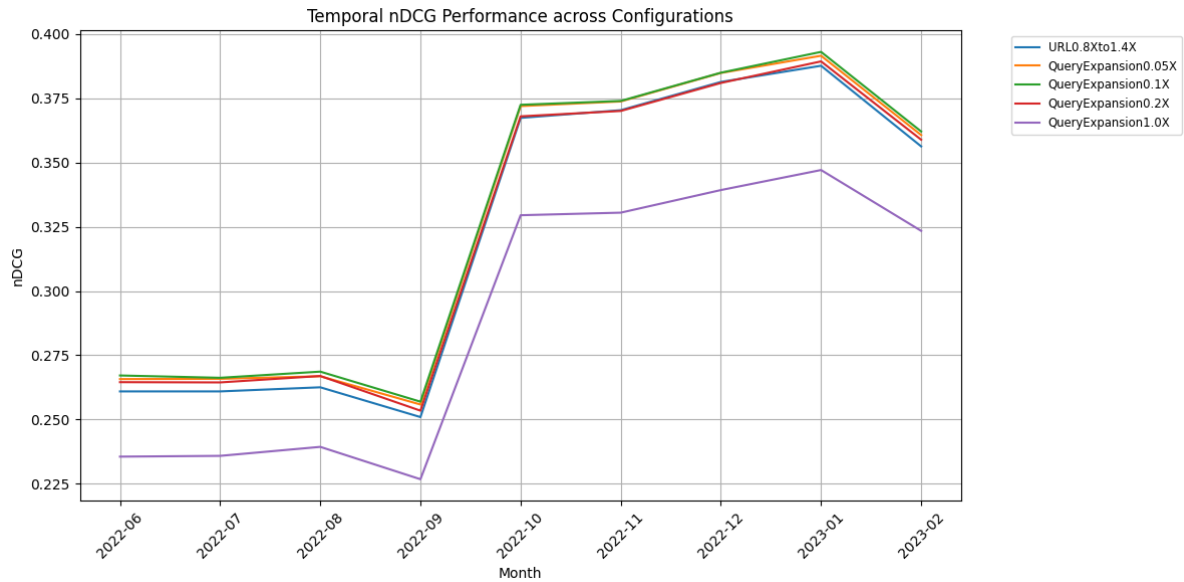


Figure 10: Average nDCG for different Query Expansion Boosting Factors

Table 4

nDCG values for previous optimal pipeline vs previous optimal pipeline + different query expansion multipliers

	2022-06	2022-07	2022-08	2022-09	2022-10	2022-11	2022-12	2023-01	2023-02
URL0.8Xto1.4X	0.2609	0.2609	0.2625	0.2509	0.3674	0.3704	0.3814	0.3877	0.3563
+QueryExpansion0.05X	0.2657	0.2658	0.2668	0.2558	0.3720	0.3738	0.3849	0.3916	0.3607
+QueryExpansion0.1X	0.2671	0.2662	0.2686	0.2569	0.3725	0.3740	0.3850	0.3931	0.3621
+QueryExpansion0.2X	0.2645	0.2644	0.2669	0.2534	0.3680	0.3701	0.3810	0.3894	0.3589
+QueryExpansion1.0X	0.2355	0.2358	0.2393	0.2267	0.3295	0.3305	0.3393	0.3471	0.3234

**Figure 11:** nDCG for different QueryExpansion Boosting Factors for each month

4.5. Query Translation and Reranking

We therefore attempted to integrate the QueryTranslation module into the pipeline; however, this did not produce any improvements and even led to a decrease in the score for certain months, as shown in Table 5. As a result, we decided to exclude this module.

As for the re-ranking module, once integrated into the pipeline, it caused a substantial decrease in the overall system performance, ultimately making its removal necessary.

Table 5

nDCG reduction with Query Translation.

	2022-06	2022-07	2022-08	2022-09	2022-10	2022-11	2022-12	2023-01	2023-02
+QueryExpansion0.1X	0.2671	0.2662	0.2686	0.2569	0.3725	0.3740	0.3850	0.3931	0.3621
+QueryTranslation0.1X	0.2669	0.2662	0.2684	0.2569	0.3724	0.3740	0.3850	0.3931	0.3620

4.6. Final Results

At this stage, we compared the pipeline step by step, obtaining the following results:

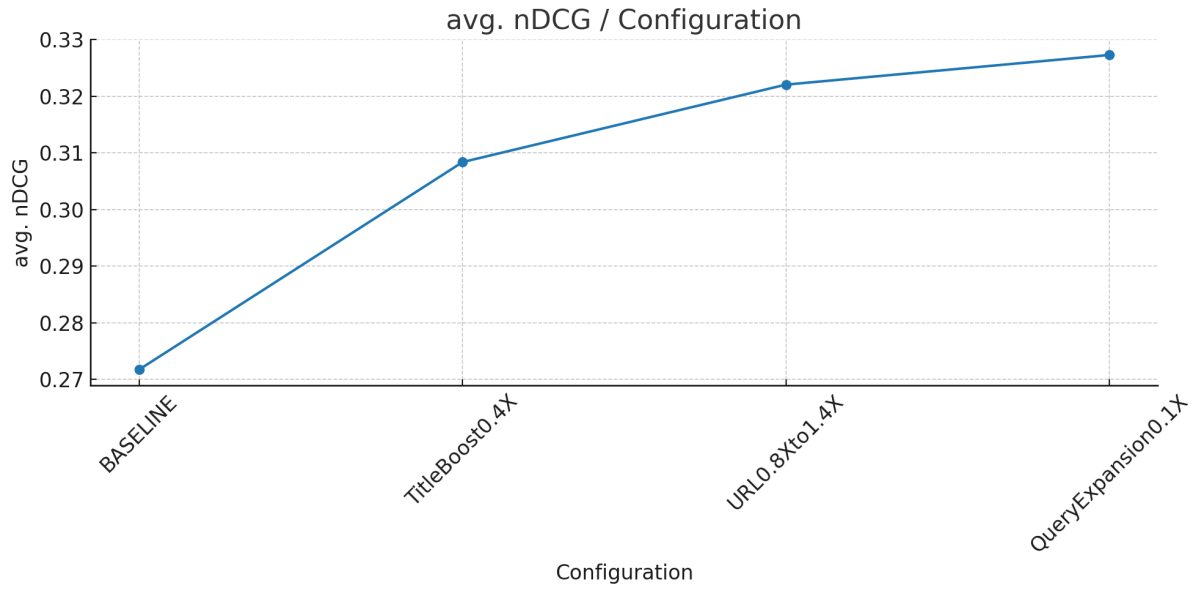


Figure 12: Average nDCG for each pipeline insertion.

Table 6
nDCG values for each optimal block configuration.

	2022-06	2022-07	2022-08	2022-09	2022-10	2022-11	2022-12	2023-01	2023-02
BASELINE	0.2107	0.2103	0.2200	0.2105	0.3142	0.3168	0.3279	0.3339	0.3014
+TitleBoost0.4X	0.2451	0.2468	0.2499	0.2405	0.3571	0.3573	0.3664	0.3743	0.3379
+URL0.8Xto1.4X	0.2609	0.2609	0.2625	0.2509	0.3674	0.3704	0.3814	0.3877	0.3563
+QueryExpansion0.1X	0.2671	0.2662	0.2686	0.2569	0.3725	0.3740	0.3850	0.3931	0.3621

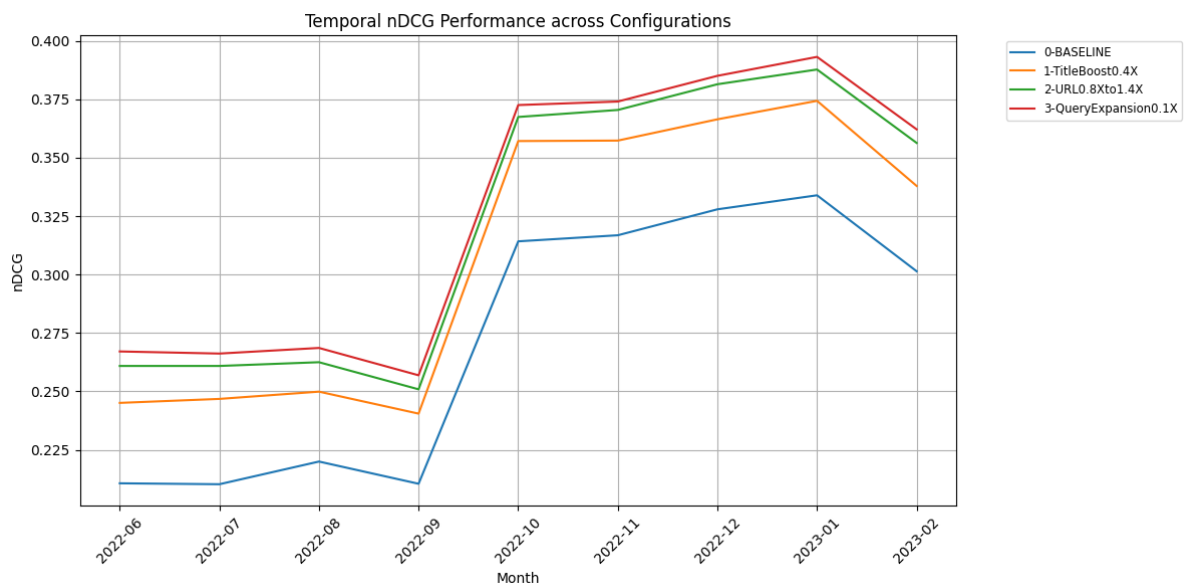


Figure 13: nDCG for each pipeline insertion in each month

5. Conclusions and Future Work

We began our project by creating a basic working structure, referred to as the baseline, which implemented a simple version of our pipeline: Parsing–Analyzer–Indexer–Searcher. From this baseline, we iteratively introduced and tested various enhancements to improve performance. These included integrating custom stopwords, implementing title boosting, applying URL alignment boosting, expanding queries using Google Gemini, and reranking results. Each component and configuration was evaluated using the *nDCG* metric to determine its impact. While the results were promising, we observed that improvements in query quality could further enhance outcomes. Future efforts could focus on refining query strategies and exploring additional optimization techniques, like:

- Trying to use n-grams: by incorporating n-grams, the system can capture contextual relationships between consecutive words rather than just individual words. This may improve search accuracy, especially when dealing with multiple terms.
- Using NER algorithms: allow the system to identify and categorize entities within a text, such as company’s name, locations, dates, phone numbers (and relative alignment for numbers with and without country code) and other important terms. By assigning higher weight to these entities, the retrieval system can give higher or lower rank results based on the significance of these keywords.
- Test other cross-encoders, more suited for multilingual tasks, like BAAI/bge-reranker-v2-m3.
- Translating all documents into French, if they are not already: translating all documents into French ensures that all documents are processed uniformly, allowing the system to apply consistent tokenization and linguistic rules. This can lead to better matching of queries with documents, but it may also introduce translation errors or nuances that could affect the retrieval quality. However, as described in the relative section, this approach would require significant computational resources and time, especially for large datasets.

This work has been based on the queries and qrels provided by the organizers as of 23rd of April 2025, at commit f759464 of the [clef-longeval/clef-longeval.github.io](https://github.com/clef-longeval/clef-longeval) repository. Such qrels have been reported as incorrect and have been changed subsequently. We did not have the chance to test our system with the new qrels, so we cannot guarantee that our results are still valid. We foresee that reranking results may be improved by using correct qrels.

References

- [1] J. M. R. N. S. G. Z. Isil Atabek, Huimin Chen, Seupd@cleftask1: Information retrieval for english and french documents: Team jihuming, 2023. URL: <https://ceur-ws.org/Vol-3497/paper-185.pdf>.
- [2] G. C. A. P. P. R. G. Z. Simone Bortolin, Gioele Ceccon, Seupd@clef: Team neon, 2023. URL: <https://ceur-ws.org/Vol-3497/paper-189.pdf>.
- [3] Y. Q. L. V. P. L. Saverio Fincato, Enrico D’Alberton, J. Arjun, Seupd@clef: Team qevals, 2023. URL: <https://ceur-ws.org/Vol-3497/paper-194.pdf>.
- [4] M. S. R. Z. N. F. Filippo Galli, Manuel Rigobello, Seupd@clef: Team iris on temporal evolution of query expansion and rank fusion techniques applied to cross-encoder re-rankers, in: Conference and Labs of the Evaluation Forum, 2024. URL: <https://ceur-ws.org/Vol-3740/paper-218.pdf>.
- [5] F. F. L. P. S. P. N. F. Lorenzo Cazzador, Francesco Luigi De Faveri, Seupd@clef: Team mouse on enhancing search engines effectiveness with large language models., in: Conference and Labs of the Evaluation Forum, 2024. URL: <https://ceur-ws.org/Vol-3740/paper-217.pdf>.
- [6] J. Savoy, Light Stemming Approaches for the French, Portuguese, German and Hungarian Languages, in: H. M. Haddad, K. M. Liebrock, R. Chbeir, M. J. Palakal, S. Ossowski, K. Yetongnoon, R. L. Wainwright, C. Nicolle (Eds.), Proc. 21st ACM Symposium on Applied Computing (SAC 2006), ACM Press, New York, USA, 2006, pp. 1031–1035.

- [7] G. Salton, M. E. Lesk, The smart automatic document retrieval systems—an illustration, *Commun. ACM* 8 (1965) 391–398. URL: <https://doi.org/10.1145/364955.364990>. doi:10.1145/364955.364990.
- [8] y. Ian Soboroff, Craig Macdonald, *usnistgov/trec_eval*, 2023. URL: https://github.com/usnistgov/trec_eval.

6. Group Members Contribution

Davide Furlan He was responsible to test stemmers vs lemmatizer, implemented the indexer, test query expansion.

Giulia Gibellato She was responsible for selecting the most suitable tokenizer based on prior research, handling language detection, initiating query translation and developing the re-ranking mechanism.

SeyedehSara NaziriAlhashem She was responsible for implementing various filters in the analyzer, including phone number and date normalization, removal of short digits, and the creation of a customized stoplist. On the search side, she contributed to the development of the *RiseSearcher* class by adding features such as title boosting and verbose flags.

Emanuele Pase He was responsible for investigating solutions in other projects, URL extraction with the interaction with the database and query expansion.

Alberto Pasqualetto He was responsible for integrating the entire pipeline and implemented the dynamic analyzer construction based on JSON configuration files. He also contributed to the implementation and execution of query expansion and translation modules, and participated in developing the parallelized database access mechanism. Additionally, he handled title extraction from parsed documents and worked on the deduplication of the qrels.

In addition to his technical contributions, he coordinated the group, ensuring that all members were actively involved and that the project progressed in a coherent and structured manner.

Filippo Tiberio He was responsible for implementing the concrete realizations of the proposed parsers and evaluating their relative efficiency. He also handled the parallelization of both the indexer and the searcher, optimizing their performance to enable the execution of as many runs as possible in the shortest amount of time. In addition, he integrated URL alignment-based reranking into the searcher.

Beyond these tasks, he managed the entire experimental pipeline, conducting all tests directly in order to fine-tune its parameters (e.g., best analyzer, title boosting, URL boosting, etc.).